

Module 9: Functions

Duration

1 Lecture + 2–3 Laboratory Hours

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of functions in PL/SQL.
 - Create and execute user-defined functions.
 - Return values from functions.
 - Differentiate between procedures and functions.
 - Invoke functions from PL/SQL blocks and SQL statements.
 - Develop reusable functions for business applications.
-

9.1 Introduction to Functions

What is a Function?

A **Function** is a named PL/SQL block that performs a specific task and **always returns a value** to the calling program.

Unlike an anonymous block, a function can be stored permanently in the Oracle database and reused whenever required.

Functions help in:

- Code reusability
 - Modular programming
 - Better maintenance
 - Faster application development
-

Real-World Examples

Functions are commonly used to:

- Calculate GST

- Calculate Employee Bonus
 - Find Student Grade
 - Calculate Income Tax
 - Determine Loan Eligibility
 - Calculate Age
 - Find Factorial
 - Calculate Simple Interest
-

Syntax of a Function

```
CREATE OR REPLACE FUNCTION function_name
(
    parameter datatype
)
RETURN datatype
IS
    -- Variable Declaration
BEGIN
    -- Business Logic
    RETURN value;
END;
/
```

Example 1: Simple Function

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION welcome_message
RETURN VARCHAR2
IS
BEGIN
    RETURN 'Welcome to Oracle PL/SQL';
END;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE(welcome_message);
END;
/
```

Output

Welcome to Oracle PL/SQL

Components of a Function

Component	Description
CREATE FUNCTION	Creates the function
Parameters	Input values (optional)
RETURN	Specifies the return datatype
BEGIN	Start of executable section
RETURN Statement	Sends value back to caller
END	End of function

Calling a Function

A function can be called in three ways.

1. Inside a PL/SQL Block

```
DECLARE

result NUMBER;

BEGIN

result:=addition(10,20);

DBMS_OUTPUT.PUT_LINE(result);

END;
/
```

2. Inside a SQL Query

```
SELECT annual_salary(50000)

FROM dual;
```

3. Assignment Statement

```
salary:=calculate_salary(20000);
```

9.2 Returning Values from Functions

A function **must return exactly one value** using the `RETURN` statement.

Syntax

```
RETURN expression;
```

Example

```
RETURN total_salary;
```

Example: Square Function

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION square
(
  number1 NUMBER
)
RETURN NUMBER
IS
BEGIN
  RETURN number1*number1;
END;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE
  (
    'Square = ' ||
    square(12)
  );
END;
/
```

Output

```
Square = 144
```

Multiple Return Statements

A function may contain multiple `RETURN` statements inside conditional structures.

Example

```
IF marks >= 40 THEN
RETURN 'PASS';
ELSE
RETURN 'FAIL';
END IF;
```

Advantages of Functions

- Modular programming
 - Code reuse
 - Easy debugging
 - Faster execution
 - Better readability
 - Can be called from SQL queries
 - Reduces code duplication
-

Limitations

- Must return only one value.
 - Cannot directly perform transaction control (`COMMIT/ROLLBACK`) when invoked from SQL.
 - Cannot return multiple independent values like procedures with `OUT` parameters.
-

9.3 Difference between Procedure and Function

Feature	Procedure	Function
Returns Value	Optional	Mandatory
RETURN Statement	Not compulsory	Compulsory
Can be called from SQL	No	Yes
Parameters	IN, OUT, IN OUT	Mostly IN
Purpose	Performs an operation	Performs a calculation and returns a result

Feature	Procedure	Function
Used For	Insert, Update, Delete	Calculations, validation, business logic
Invocation	EXEC or PL/SQL Block	SQL Query or PL/SQL Block

Example Comparison

Procedure

```
CREATE OR REPLACE PROCEDURE display_name  
  
IS  
  
BEGIN  
  
DBMS_OUTPUT.PUT_LINE('Oracle');  
  
END;  
/
```

Function

```
CREATE OR REPLACE FUNCTION display_name  
  
RETURN VARCHAR2  
  
IS  
  
BEGIN  
  
RETURN 'Oracle';  
  
END;  
/
```

When to Use a Procedure?

Use procedures when:

- Updating employee salary
 - Inserting records
 - Deleting records
 - Banking transactions
 - Payroll processing
 - Library management
-

When to Use a Function?

Use functions when:

- Calculating GST
 - Finding age
 - Calculating bonus
 - Determining grades
 - Checking prime numbers
 - Calculating factorial
 - Computing interest
-

Real-Time Applications

Functions are widely used in:

- Banking Software
 - Payroll Systems
 - Hospital Management
 - Insurance Applications
 - ERP Systems
 - Student Information Systems
 - Railway Reservation
 - Library Management
 - E-Commerce Websites
 - Human Resource Management
-

Summary

A **Function** in PL/SQL is a reusable program unit that performs a specific task and **returns a single value**. Functions simplify application development by encapsulating business logic into reusable modules. They can be called from SQL queries as well as PL/SQL blocks, making them ideal for calculations, validations, and data transformations. Understanding the distinction between procedures and functions is essential for developing efficient Oracle database applications.

Laboratory Programs

Program 1: GST Calculator Function

Aim

To calculate GST at 18% using a user-defined function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION calculate_gst
(
    amount NUMBER
)
RETURN NUMBER
IS
BEGIN
    RETURN amount * 0.18;
END;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE
    (
        'GST Amount = ' ||
        calculate_gst(25000)
    );
END;
/
```

Output

```
GST Amount = 4500
```

Program 2: Age Calculator Function

Aim

To calculate the age of a person from the date of birth.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION calculate_age
(
    dob DATE
)
RETURN NUMBER
IS
BEGIN
    RETURN FLOOR(MONTHS_BETWEEN(SYSDATE,dob)/12);
END;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE
    (
        'Age = ' ||
        calculate_age(TO_DATE('15-08-2003','DD-MM-YYYY')) ||
        ' Years'
    );
END;
/
```

Output

Age = 22 Years

(Output will vary depending on the current system date.)

Program 3: Factorial Function

Aim

To calculate the factorial of a given number using a function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION factorial
(
    n NUMBER
```

```

)

RETURN NUMBER

IS

    fact NUMBER := 1;

BEGIN

    FOR i IN 1..n LOOP

        fact := fact * i;

    END LOOP;

    RETURN fact;

END;
/

BEGIN

    DBMS_OUTPUT.PUT_LINE
    (
        'Factorial = ' ||
        factorial(5)
    );

END;
/

```

Output

```
Factorial = 120
```

Additional Practice Programs

1. Function to calculate Simple Interest.
 2. Function to calculate Compound Interest.
 3. Function to determine whether a number is Even or Odd.
 4. Function to check whether a number is Prime.
 5. Function to reverse a number.
 6. Function to check whether a number is a Palindrome.
 7. Function to calculate Income Tax.
 8. Function to calculate Employee Bonus.
 9. Function to find the Largest of Three Numbers.
 10. Function to calculate Annual Salary.
-

Viva Questions

1. What is a function in PL/SQL?
2. Why must every function contain a `RETURN` statement?
3. What is the difference between a procedure and a function?
4. Can a function be called from an SQL statement? Explain with an example.
5. What are the advantages of using functions?
6. What types of parameters are commonly used in functions?
7. What is the purpose of the `RETURN` clause in a function declaration?
8. Can a function call another function or procedure?
9. In which situations is a function preferred over a procedure?
10. Give three real-world applications where PL/SQL functions are commonly used.